

Python语言与系统设计

史骏

juns@hfut.edu.cn

内容

01 程序的控制结构

02 函数

03 组合数据类型

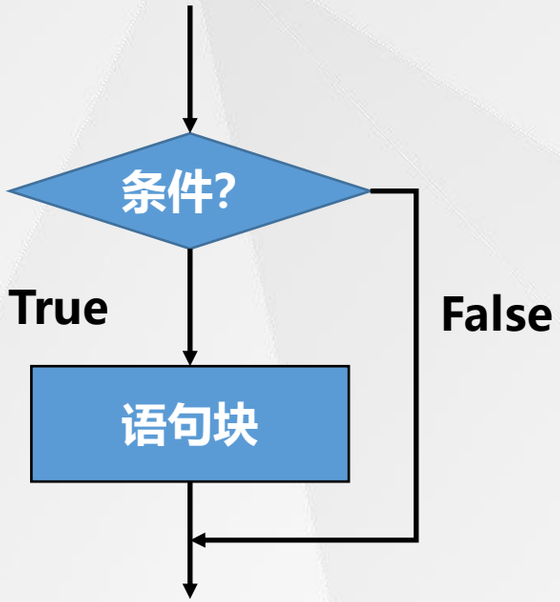
04 文件

01 程序的控制结构

单分支结构

根据判断条件结果而选择不同向前路径的运行方式

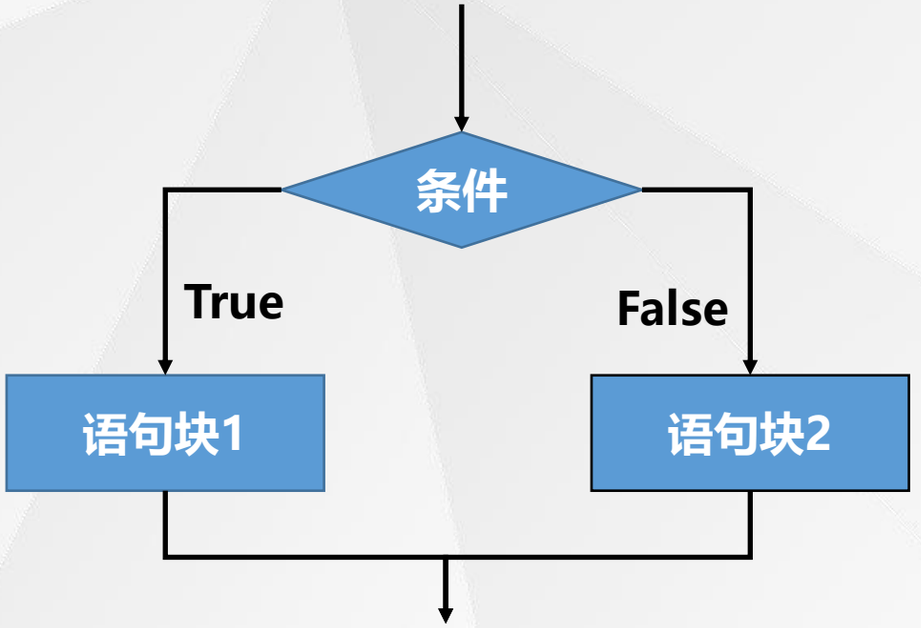
```
if <条件>：
    <语句块>
```



二分支结构

根据判断条件结果而选择不同向前路径的运行方式

```
if <条件>：
    <语句块1>
else：
    <语句块2>
```



二分支结构

紧凑形式：适用于简单表达式的二分支结构

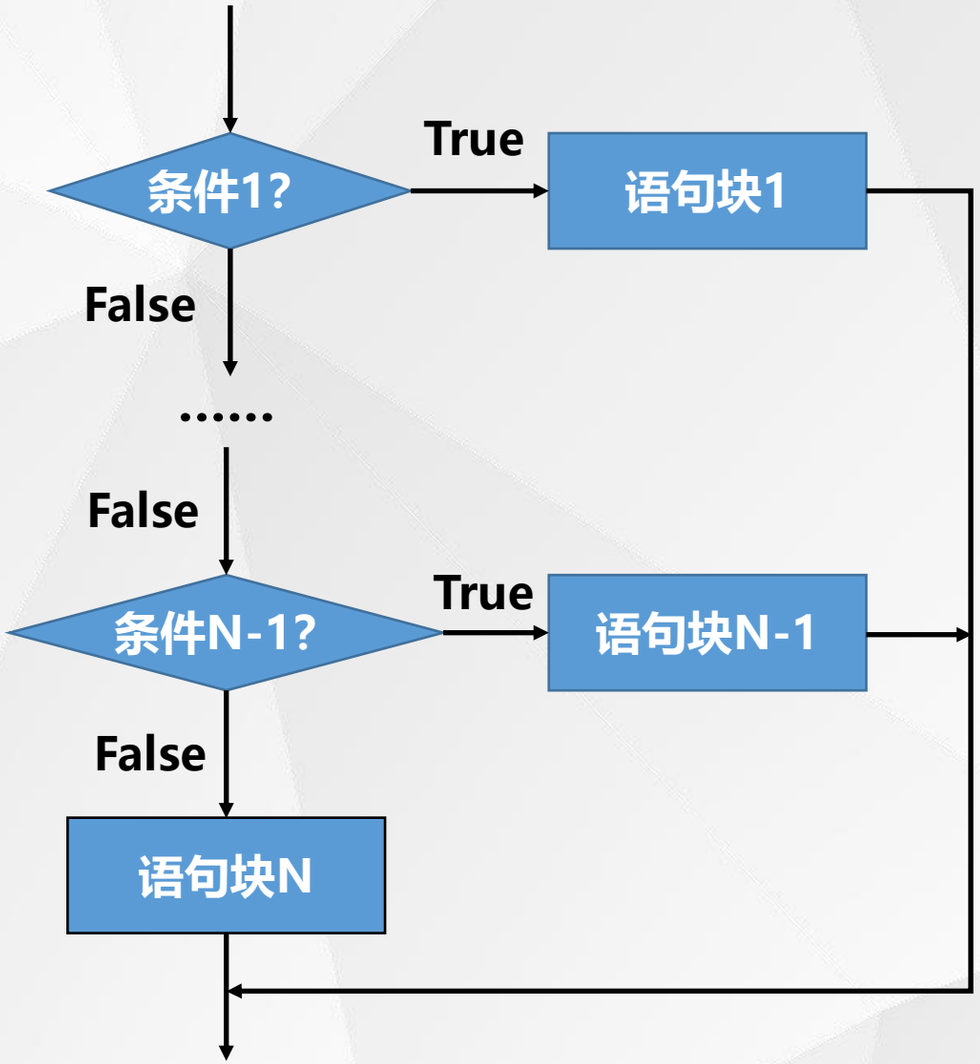
<表达式1> if <条件> else <表达式2>

```
1    guess = eval(input())
2    print("猜{}了".format("对" if guess==99 else "错"))
```

```
PS D:\test>  & 'C:\Users\Lenovo\AppData\
on-2020.11.367453362\pythonFiles\lib\pyt
222
猜错了
PS D:\test>  cd 'd:\test'; & 'C:\Users\L
ms-python.python-2020.11.367453362\pytho
99
猜对了
PS D:\test>
```

多分支结构

```
if  <条件1>：
    <语句块1>
elif  <条件2>：
    <语句块2>
    ……
else：
    <语句块N>
```



条件判断

操作符	描述
==	等于，结果True或者False
!=	不等于，结果True或者False
>	大于，结果True或者False
<	小于，结果True或者False
>=	大于等于，结果True或者False
<=	小于等于，结果True或者False

条件组合

用于条件组合的三个保留字

运算符	表达式	描述
and	x **and** y	两个条件x和y的逻辑**与**
or	x **or** y	两个条件x和y的逻辑**或**
not	**not** x	条件x的逻辑**非**

程序的异常处理

```
1    num = eval(input("请输入一个数字: "))
2    print(num**2)
```

```
PS D:\test>  cd 'd:\test'; & 'C:\Users\Lenovo\A
ms-python.python-2020.11.367453362\pythonFiles\
请输入一个数字: *
Traceback (most recent call last):
  File "d:\test\test.py", line 1, in <module>
    num = eval(input("请输入一个数字: "))
  File "<string>", line 1
    *
    ^
SyntaxError: unexpected EOF while parsing
```

文件名

异常发生的代码行数

异常类型

异常内容提示

程序的异常处理

异常处理的基本使用

```
try：
    <语句块1>
except：
    <语句块2>
```

```
try：
    <语句块1>
except <异常类型>：
    <语句块2>
```

程序的异常处理

异常名称	描述
IOError	输入/输出操作失败
IndexError	序列中没有此索引(index)
MemoryError	内存溢出错误
ZeroDivisionError	除(或取模)零 (所有数据类型)
AttributeError	对象没有这个属性
NameError	未声明/初始化对象 (没有属性)
SyntaxError	Python 语法错误
TabError	Tab 和空格混用
ValueError	传入无效的参数

遍历循环

for <循环变量> in <遍历结构>:

<语句块>

- 计数循环(N次)
- 字符串遍历循环
- 文件遍历循环
- 计数循环(特定次)
- 列表遍历循环
- ……

遍历循环的应用

计数循环（N次）

```
for i in range(N):
    <语句块>
```

- 遍历由range()函数产生的数字序列，产生循环

遍历循环的应用

计数循环（特定次）

```
for i in range(M, N, K):
    <语句块>
```

- 遍历由range()函数产生的数字序列，产生循环

遍历循环的应用

字符串遍历循环

```
for c in s:
    <语句块>
```

- s是字符串，遍历字符串每个字符，产生循环

遍历循环的应用

列表遍历循环

```
for item in ls:
    <语句块>
```

- ls是一个列表， 遍历其每个元素，产生循环

遍历循环的应用

文件遍历循环

```
for line in fi:
    <语句块>
```

- fi是一个文件标识符，遍历其每行，产生循环

while循环

由条件控制的循环运行方式

while <条件>：

<语句块>

- 反复执行语句块，直到条件不满足时结束

循环控制保留字

break & continue

- break跳出并结束当前整个循环，执行循环后的语句
- continue结束当次循环，继续执行后续次数循环
- break和continue可以与for和while循环搭配使用

02

函数

函数

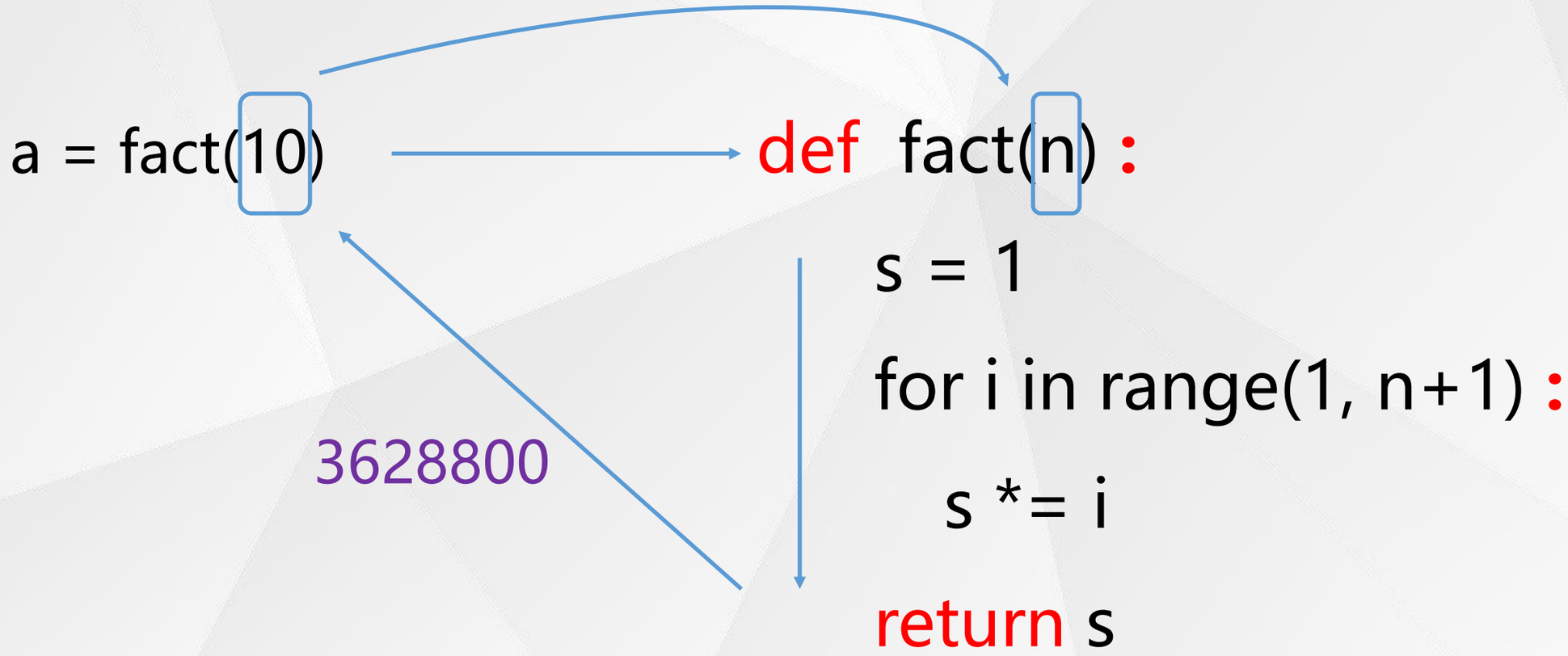
- 函数的定义
- 函数的使用及调用过程
- 函数的参数传递
- 函数的返回值
- 局部变量和全局变量
- lambda函数

函数的定义

函数是一段代码的表示

```
def  <函数名>(<参数(0个或多个)>):
    <函数体>
    return <返回值>
```

函数的调用过程



可选参数传递

函数定义时可以为某些参数指定默认值，构成可选参数

```
def  <函数名>(<非可选参数>,<可选参数>):
     <函数体>
     return <返回值>
```

可变参数传递

函数定义时可以设计可变数量参数，即不确定参数总数量

```
def  <函数名>(<参数>, *b) :
    <函数体>
    return <返回值>
```

参数传递的两种方式

函数调用时，参数可以按照位置或名称方式传递

```
def  fact(n, m=1) :
    s = 1
    for i in range(1, n+1):
        s *= i
    return s//m
```

```
>>>fact(10)
3628800
>>>fact(10, 5)
725760
>>>fact(m=5, n=10)
725760
```

位置传递

名称传递

函数的返回值

函数可以返回0个或多个结果

- return保留字用来传递返回值
- 函数可以有返回值，也可以没有，可以有return，也可以没有
- return可以传递0个返回值，也可以传递任意多个返回值

局部变量和全局变量

程序

全局变量

<语句块1>

def <函数名>(<参数>):

<函数体>

return <返回值>

<语句块2>

函数

局部变量

Lambda函数

<函数名> = lambda <参数>: <表达式>



def <函数名>(<参数>) :

<函数体>

return <返回值>

03

组合数据类型

组合数据类型

- 集合类型及操作
- 序列类型及操作
- 字典类型及操作

序列类型及操作 → 元组类型、列表类型

集合类型的定义

集合是多个元素的无序组合

- 集合类型与数学中的集合概念一致
- 集合元素之间无序，每个元素唯一，不存在相同元素
- 集合元素不可更改，不能是可变数据类型

集合类型的定义

可变数据类型与不可变数据类型

- 不可变数据类型：当该数据类型的对应变量的值发生了改变，它对应的内存地址也会发生改变
- 可变数据类型：当该数据类型的对应变量的值发生了改变，它对应的内存地址不发生改变
- 不可变数据类型更改后地址发生改变，可变数据类型更改地址不发生改变

数据类型	可变/不可变
整型	不可变
字符串	不可变
元组	不可变
列表	可变
集合	可变
字典	可变

集合类型的定义

集合是多个元素的无序组合

- 集合用大括号 {} 表示，元素间用逗号分隔
- 建立集合类型用 {} 或 set()
- 建立空集合类型，必须使用set()

集合操作符

操作符及应用	描述
S | T	并，返回一个新集合，包括在集合S和T中的所有元素
S - T	差，返回一个新集合，包括在集合S但不在T中的元素
S & T	交，返回一个新集合，包括同时在集合S和T中的元素
S ^ T	补，返回一个新集合，包括集合S和T中的非相同元素
S <= T 或 S < T	返回True/False，判断S和T的子集关系
S >= T 或 S > T	返回True/False，判断S和T的包含关系

集合操作符

操作符及应用	描述
S |= T	补，更新集合S，包括集合S和T中的非相同元素
S -= T	差，更新集合S，包括在集合S但不在T中的元素
S &= T	交，更新集合S，包括同时在集合S和T中的元素
S ^= T	补，更新集合S，包括集合S和T中的非相同元素

集合处理方法

操作函数或方法	描述
S.add(x)	如果x不在集合S中，将x增加到S
S.discard(x)	移除S中元素x，如果x不在集合S中，不报错
S.remove(x)	移除S中元素x，如果x不在集合S中，产生KeyError异常
S.clear()	移除S中所有元素
S.pop()	随机返回S的一个元素，更新S，若S为空产生KeyError异常

集合处理方法

操作函数或方法	描述
S.copy()	返回集合S的一个副本
len(S)	返回集合S的元素个数
x in S	判断S中元素x， x在集合S中，返回True，否则返回False
x not in S	判断S中元素x， x不在集合S中，返回True，否则返回False
set(x)	将其他类型变量x转变为集合类型

组合数据类型

- 集合类型及操作
- 序列类型及操作
- 字典类型及操作

元组类型

列表类型

序列类型的定义

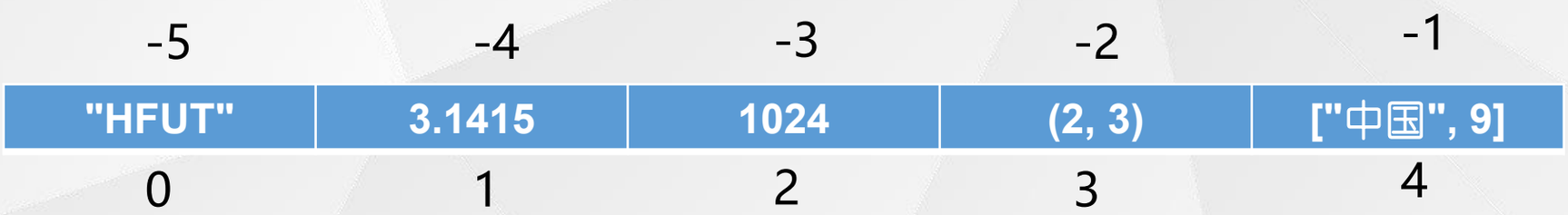
序列是一个基类类型



序列类型定义

序号的定义

反向递减序号



正向递增序号

序列类型通用操作符

操作符及应用	描述
x in s	如果x是序列s的元素，返回True，否则返回False
x not in s	如果x是序列s的元素，返回False，否则返回True
s + t	连接两个序列s和t
s*n 或 n*s	将序列s复制n次
s[i]	索引，返回s中的第i个元素， i是序列的序号
s[i: j] 或 s[i: j: k]	切片，返回序列s中第i到j以k为步长的元素子序列

元组类型定义

元组是序列类型的一种扩展

- 元组是一种序列类型，一旦创建就不能被修改
- 使用小括号 () 或 tuple() 创建，元素间用逗号 , 分隔
- 可以使用或不使用小括号

```
>>> creature = "cat", "dog", "tiger", "human"
>>> creature
('cat', 'dog', 'tiger', 'human')
```

列表类型定义

列表是序列类型的一种扩展

- 列表是一种序列类型，创建后可以随意被修改
- 使用方括号 [] 或list() 创建，元素间用逗号，分隔
- 列表中各元素类型可以不同，无长度限制

列表类型操作函数和方法

函数或方法	描述
ls[i] = x	替换列表ls第i元素为x
ls[i: j: k] = lt	用列表lt替换ls切片后所对应元素子列表
del ls[i]	删除列表ls中第i元素
del ls[i: j: k]	删除列表ls中第i到第j以k为步长的元素
ls += lt	更新列表ls，将列表lt元素增加到列表ls中
ls *= n	更新列表ls，其元素重复n次

列表类型操作函数和方法

函数或方法	描述
ls.append(x)	在列表ls最后增加一个元素x
ls.clear()	删除列表ls中所有元素
ls.copy()	生成一个新列表，赋值ls中所有元素
ls.insert(i,x)	在列表ls的第i位置增加元素x
ls.pop(i)	将列表ls中第i位置元素取出并删除该元素
ls.remove(x)	将列表ls中出现的第一个元素x删除
ls.reverse()	将列表ls中的元素反转

字典类型定义

字典类型是“映射”的体现

- 键值对：键是数据索引的扩展
- 字典是键值对的集合，键值对之间无序
- 采用大括号{}和dict()创建，键值对用冒号: 表示

{<键1>:<值1>, <键2>:<值2>, ... , <键n>:<值n>}

字典类型的用法

在字典变量中，通过键获得值

字典变量={<键1>:<值1>, <键2>:<值2>, … , <键n>:<值n>}

<值> = <字典变量>[<键>]　　<字典变量>[<键>] = <值>

[ ] 用来向字典变量中索引或增加元素

字典类型操作函数和方法

函数或方法	描述
del d[k]	删除字典d中键k对应的数据值
k in d	判断键k是否在字典d中，如果在返回True，否则False
d.keys()	返回字典d中所有的键信息
d.values()	返回字典d中所有的值信息
d.items()	返回字典d中所有的键值对信息
del d[k]	删除字典d中键k对应的数据值

字典类型操作函数和方法

函数或方法	描述
d.get(k, <default>)	键k存在，则返回相应值，不在则返回<default>值
d.pop(k, <default>)	键k存在，则取出相应值，不在则返回<default>值
d.popitem()	随机从字典d中取出一个键值对，以元组形式返回
d.clear()	删除所有的键值对
len(d)	返回字典d中元素的个数

Python基本数据类型

- 数字类型
- 字符串
- 列表
- 元组
- 集合
- 字典

序列

04 文件

文件

- 文件的类型
- 文件的打开和关闭
- 文件内容的读取
- 数据的文件写入
- 一维数据的格式化处理
- 二维数据的格式化处理

文件的理解

文本文件 vs. 二进制文件

- 文件文件和二进制文件只是文件的展示方式
- 本质上，所有文件都是二进制形式存储
- 形式上， 所有文件采用两种方式展示

文本文件 vs. 二进制文件

"中国是个伟大的国家!"

- 文本形式

中国是个伟大的国家!

- 二进制形式

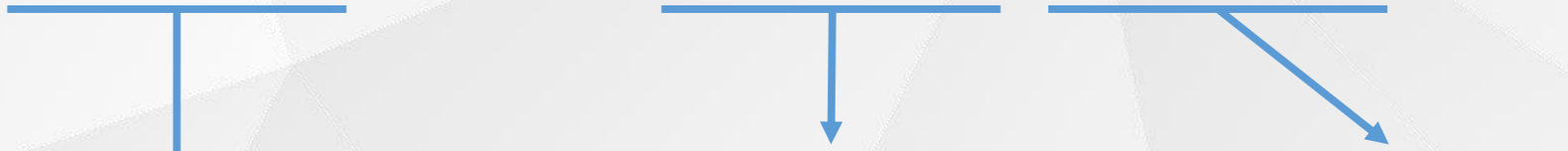
b'\xd6\xd0\xb9\xfa\xca\xc7\xb8\xf6\xce\xb0\xb4\xf3\xb5\
xc4\xb9\xfa\xbc\xd2\xa3\xa1'

文件

- 文件的类型
- 文件的打开和关闭
- 文件内容的读取
- 数据的文件写入
- 一维数据的格式化处理
- 二维数据的格式化处理

文件的打开

<变量名> = open(<文件名>, <打开模式>)



文件句柄

文件路径和名称

源文件同目录可省路径

文本 or 二进制

读 or 写

打开模式

f = open("f.txt")	- 文本形式、只读模式、默认值
f = open("f.txt", "rt")	- 文本形式、只读模式、同默认值
f = open("f.txt", "w")	- 文本形式、覆盖写模式
f = open("f.txt", "a+")	- 文本形式、追加写模式+ 读文件
f = open("f.txt", "x")	- 文本形式、创建写模式
f = open("f.txt", "b")	- 二进制形式、只读模式
f = open("f.txt", "wb")	- 二进制形式、覆盖写模式

文件的关闭

<变量名> .close()



文件句柄

文件使用

```
#文本形式打开文件
tf = open("f.txt", "rt")
print(tf.readline())
tf.close()
```

```
#二进制形式打开文件
bf = open("f.txt", "rb")
print(bf.readline())
bf.close()
```

文件内容的读取

操作方法	描述
<f>.read(size=-1)	读入全部内容，如果给出参数，读入前size长度 >>>s = f.read(2) 中国
<f>.readline(size=-1)	读入一行内容，如果给出参数，读入该行前size长度 >>>s = f.readline() 中国是一个伟大的国家！

文件内容的读取

操作方法	描述
<f>.readlines(hint=-1)	读入文件所有行，以每行为元素形成列表 如果给出参数，读入前hint行 >>>s = f.readlines() ['中国是一个伟大的国家！ ']

数据的文件写入

操作方法	描述
<f>.write(s)	向文件写入一个字符串或字节流 >>>f.write("中国是一个伟大的国家!")
<f>.writelines(lines)	将一个元素全为字符串的列表写入文件 >>>ls = ["中国", "法国", "美国"] >>>f.writelines(ls) 中国法国美国

数据的文件写入

操作方法	描述
<f>.seek(offset)	改变当前文件操作指针的位置， offset含义如下： 0 – 文件开头； 1 – 当前位置； 2 – 文件结尾 >>>f.seek(0) #回到文件开头

一维数据的格式化处理

- 数据的维度：一维、二维、多维、高维
- 一维数据的表示：列表类型(有序)和集合类型(无序)
- 一维数据的存储：空格分隔、逗号分隔、特殊符号分隔
- 一维数据的处理：字符串方法 .split() 和 .join()

二维数据的格式化处理

- 二维数据的表示：列表类型，其中每个元素也是一个列表
- CSV格式：逗号分隔表示一维，按行分隔表示二维
- JSON数据：JSON对象和数组
- 二维数据的处理： for循环+.split()和.join()



史骏
juns@hfut.edu.cn